

app\front_led_management.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      front_led_management.py
4  @brief     Gestionnaire de LED frontale avec patterns d'erreur.
5  @details   Ce module fournit une classe pour gérer les clignotements et états
6             de la LED frontale avec différents patterns d'erreur. Supporte
7             les modes continus, les clignotements d'erreur et la gestion
8             thread-safe des patterns LED.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Gestion multithread pour les patterns asynchrones
20 import threading
21 # Fonctions temporelles pour les délais et temporisations
22 import time
23 # Abstraction GPIO (bibliothèque periphery)
24 from periphery import GPIO
25
26 # -----
27 # Classes principales
28 # -----
29
30 class LedErrorIndicator:
31     """
32     @brief     Gestionnaire de LED avec patterns d'erreur.
33     @details   Classe pour contrôler une LED frontale avec différents patterns
34               de clignotement selon les états du système. Supporte les modes
35               normal, standby, séquence en cours et divers patterns d'erreur
36               avec clignotements codifiés.
37     """
38
39     # Définition des patterns d'erreur (temps de base pour clignotement)
40     # 200ms ON/OFF pour un clignotement plus rapide [s]
41     BASE_BLINK_TIME = 0.2
42     # Cycle de 10 secondes [s]
43     CYCLE_TIME = 10.0
44
45     # Dictionnaire des patterns disponibles
46     ERROR_PATTERNS = {
47         # LED normalement allumée en continu
48         'normal': 'solid_on',
49
50         # LED éteinte (standby)
51         'standby': 'solid_off',
52
53         # Clignotement continu pendant séquence de mesure
54         'sequence_running': 'continuous_blink',
55
56         # Patterns d'erreur : X clignotements sur 10 secondes puis répétition
57         'no_internet': 1,      # 1 clignotement = Pas de connexion Internet
58         'api_failed': 2,      # 2 clignotements = Erreur API (connexion ou réponse)
59         'measure_failed': 3,   # 3 clignotements = Erreur de mesure ADC/résistance
60         'critical_error': 4,   # 4 clignotements = Erreur critique système
61         'config_error': 5     # 5 clignotements = Erreur de configuration
62     }
63
64     def __init__(self, gpio_chip_path, led_pin):
65         """
66         @brief     Initialise le gestionnaire LED.
67         @details   Configure l'interface GPIO pour la LED, initialise les variables
68                   de thread et les mécanismes de synchronisation thread-safe.
69
70         @param     gpio_chip_path Chemin vers le périphérique GPIO.
71         @param     led_pin        Numéro de pin de la LED.
72         """
73         # Initialisation de l'interface GPIO pour la LED
74         self.led = GPIO(gpio_chip_path, led_pin, "out")
75         # Pattern actuel (par défaut normal)
76         self.current_pattern = 'normal'
77         # Thread pour le pattern en cours
78         self.pattern_thread = None
79         # Événement pour arrêter le pattern
80         self.pattern_stop = threading.Event()
81         # Verrou pour la synchronisation thread-safe
82         self.led_lock = threading.Lock()
83
84     def set_pattern(self, pattern_name):

```

```

85     """
86     @brief Change le pattern de clignotement.
87     @details Arrête le pattern précédent et démarre le nouveau selon
88             le nom fourni. Gère les modes continus, clignotements
89             d'erreur et états fixes de manière thread-safe.
90
91     @param pattern_name Nom du pattern à activer.
92     """
93     # Vérification de l'existence du pattern
94     if pattern_name not in self.ERROR_PATTERNS:
95         print(f"[LED] Pattern inconnu: {pattern_name}")
96         return
97
98     # Ne rien faire si c'est déjà le pattern actuel
99     if self.current_pattern == pattern_name:
100         return
101
102     # Section critique thread-safe
103     with self.led_lock:
104         # Arrêt du pattern précédent
105         # Vérification de l'existence et de l'activité du thread
106         if self.pattern_thread and self.pattern_thread.is_alive():
107             # Demande d'arrêt du thread
108             self.pattern_stop.set()
109             # Attente de la fin du thread avec timeout
110             self.pattern_thread.join(timeout=1)
111
112         # Démarrage du nouveau pattern
113         # Stockage du nouveau pattern courant
114         self.current_pattern = pattern_name
115         # Remise à zéro de l'événement d'arrêt
116         self.pattern_stop.clear()
117
118         # Récupération de la valeur du pattern
119         pattern_value = self.ERROR_PATTERNS[pattern_name]
120
121         # Gestion des différents types de patterns
122         if pattern_value == 'solid_on':
123             # LED allumée en continu
124             self.led.write(True)
125             print(f"[LED] Pattern activé: {pattern_name} (LED allumée)")
126         elif pattern_value == 'solid_off':
127             # LED éteinte en continu
128             self.led.write(False)
129             print(f"[LED] Pattern activé: {pattern_name} (LED éteinte)")
130         elif pattern_value == 'continuous_blink':
131             # Clignotement continu
132             # Création du thread pour clignotement continu
133             self.pattern_thread = threading.Thread(target=self._continuous_blink, daemon=True)
134             # Démarrage du thread
135             self.pattern_thread.start()
136             print(f"[LED] Pattern activé: {pattern_name} (clignotement continu)")
137         elif isinstance(pattern_value, int):
138             # Pattern d'erreur avec X clignotements
139             # Création du thread pour pattern d'erreur
140             self.pattern_thread = threading.Thread(target=self._error_pattern, args=(pattern_value,), daemon=True)
141             # Démarrage du thread
142             self.pattern_thread.start()
143             print(f"[LED] Pattern activé: {pattern_name} ({pattern_value} clignotement(s) sur 10s)")
144
145     def _continuous_blink(self):
146         """
147         @brief Clignotement continu pendant séquence.
148         @details Méthode privée exécutée dans un thread séparé pour gérer
149                 le clignotement continu de la LED. Alterne entre ON/OFF
150                 avec le temps de base défini jusqu'à réception d'un signal d'arrêt.
151         """
152         # Boucle de clignotement jusqu'à demande d'arrêt
153         while not self.pattern_stop.is_set():
154             try:
155                 # Allumage de la LED
156                 self.led.write(True)
157                 # Attente avec vérification d'arrêt
158                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
159                     break
160                 # Extinction de la LED
161                 self.led.write(False)
162                 # Attente avec vérification d'arrêt
163                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
164                     break
165             except Exception as e:
166                 # Gestion des erreurs de clignotement
167                 print(f"[LED] Erreur clignotement continu: {e}")
168                 break
169
170     def _error_pattern(self, blink_count):
171         """

```

```

172     @brief Pattern d'erreur avec nombre de clignotements défini.
173     @details Méthode privée pour gérer les patterns d'erreur. Effectue
174             un nombre spécifique de clignotements puis attend jusqu'à
175             compléter un cycle de 10 secondes avant de répéter.
176
177     @param blink_count Nombre de clignotements à effectuer par cycle.
178     """
179     # Boucle de pattern d'erreur jusqu'à demande d'arrêt
180     while not self.pattern_stop.is_set():
181         try:
182             # Phase clignotements
183             # Boucle pour le nombre de clignotements demandé
184             for i in range(blink_count):
185                 # Vérification d'arrêt
186                 if self.pattern_stop.is_set():
187                     break
188                 # ON
189                 # Allumage de la LED
190                 self.led.write(True)
191                 # Attente avec vérification d'arrêt
192                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
193                     break
194                 # OFF
195                 # Extinction de la LED
196                 self.led.write(False)
197                 # Attente avec vérification d'arrêt
198                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
199                     break
200
201             # Vérification d'arrêt avant la phase d'attente
202             if self.pattern_stop.is_set():
203                 break
204
205             # Phase d'attente (9 temps vides pour compléter les 10 secondes)
206             # Temps utilisé pour clignotements : blink_count * (0.5 + 0.5) = blink_count secondes
207             # Temps restant : 10 - blink_count secondes
208             # Calcul du temps restant pour compléter le cycle
209             remaining_time = self.CYCLE_TIME - (blink_count * 2 * self.BASE_BLINK_TIME)
210             # Attente seulement si du temps reste
211             if remaining_time > 0:
212                 # LED éteinte pendant l'attente
213                 self.led.write(False)
214                 # Attente du temps restant avec vérification d'arrêt
215                 if self.pattern_stop.wait(remaining_time):
216                     break
217
218         except Exception as e:
219             # Gestion des erreurs de pattern d'erreur
220             print(f"[LED] Erreur pattern d'erreur: {e}")
221             break
222
223     def set_solid(self, state):
224         """
225         @brief LED fixe ON ou OFF.
226         @details Force la LED dans un état fixe (allumée ou éteinte) en
227                 arrétant tout pattern en cours. Utilisé pour des états
228                 temporaires ou des tests.
229
230         @param state État demandé (True=ON, False=OFF).
231         """
232         # Section critique thread-safe
233         with self.led_lock:
234             # Arrêt du thread de pattern si actif
235             if self.pattern_thread and self.pattern_thread.is_alive():
236                 # Demande d'arrêt du thread
237                 self.pattern_stop.set()
238                 # Attente de la fin du thread avec timeout
239                 self.pattern_thread.join(timeout=1)
240             try:
241                 # Application de l'état demandé
242                 self.led.write(state)
243                 # Affichage de confirmation
244                 print(f"[LED] État fixe: {'ON' if state else 'OFF'}")
245             except Exception as e:
246                 # Gestion des erreurs d'écriture GPIO
247                 print(f"[LED] Erreur état fixe: {e}")
248
249     def close(self):
250         """
251         @brief Ferme proprement la LED.
252         @details Arrête tous les threads en cours, éteint la LED et ferme
253                 l'interface GPIO. À appeler avant la fin du programme pour
254                 un nettoyage propre des ressources.
255         """
256         # Section critique thread-safe
257         with self.led_lock:
258             # Arrêt du thread de pattern si actif

```

```
259         if self.pattern_thread and self.pattern_thread.is_alive():
260             # Demande d'arrêt du thread
261             self.pattern_stop.set()
262             # Attente de la fin du thread avec timeout
263             self.pattern_thread.join(timeout=1)
264         try:
265             # Extinction de la LED
266             self.led.write(False)
267             # Fermeture de l'interface GPIO
268             self.led.close()
269             # Confirmation de fermeture
270             print("[LED] LED fermée")
271         except Exception as e:
272             # Gestion des erreurs de fermeture
273             print(f"[LED] Erreur fermeture: {e}")
274
275 # -----
276 # Fonctions utilitaires
277 # -----
278
279 def create_led_indicator(gpio_chip_path, led_pin):
280     """
281     @brief  Factory function pour créer une instance LedErrorIndicator.
282     @details Fonction utilitaire qui encapsule la création et l'initialisation
283             d'un gestionnaire LED. Gère les erreurs d'initialisation et
284             configure automatiquement le pattern normal par défaut.
285
286     @param  gpio_chip_path Chemin vers le périphérique GPIO.
287     @param  led_pin        Numéro de pin de la LED.
288
289     @return Instance de LedErrorIndicator si succès, None sinon.
290     """
291
292     try:
293         # Création de l'instance du gestionnaire LED
294         led_indicator = LedErrorIndicator(gpio_chip_path, led_pin)
295         # LED allumée par défaut
296         led_indicator.set_pattern('normal')
297         # Confirmation de l'initialisation
298         print("[LED] Gestionnaire LED initialisé avec succès")
299         return led_indicator
300     except Exception as e:
301         # Gestion des erreurs d'initialisation
302         print(f"[LED] Erreur lors de l'initialisation: {e}")
303         return None
```